VIVO SWORD Deposit

Technical Specification

SWORD-Based Deposit of Publications from VIVO to an Institutional Repository

Document Status: FINAL
Version: 0.6
Date: June 2026
Source Story: V1
Project: LYRASIS Interoperability Project
Systems: VIVO / any SWORD-enabled IR

Purpose and Scope
Background
Actors and Roles
System Overview
Integration Architecture
Behavior Scenarios
Error Scenarios
Other Requirements
Implementation Gaps

Purpose and Scope

This specification defines **how VIVO would implement** a workflow in which a signed-in researcher, viewing one of their existing **publication** pages, starts a deposit by choosing a single file, either a **PDF** or a **METS-compliant ZIP** (for example, a PDF plus a METS-formatted XML file), which VIVO deposits to a configured IR via **SWORD v2**, and then, on a successful (or pending) deposit, **adds a Website (a vcard webpage) to that publication** whose URL is the IR item URI returned by the deposit. Because the publication already exists in VIVO, no publication record is created and no publication metadata is *entered* as part of the deposit. For **PDF** deposits, VIVO reads a **small hard-coded set of fields** from the publication (e.g. `bibo:title`) and sends them with the deposit as Dublin Core via JavaClient2.0 `EntryPart.addDublinCore(...)`. For **METS ZIP** deposits, the archive is passed through unmodified (its metadata is already inside the package). Admin-configurable field mapping is out

of scope for this revision; METS is the path for advanced / custom metadata packaging. SWORD v2 is the primary protocol in scope. There are currently no known implementations of SWORD v3, but the spec notes where SWORD v3 configuration would differ from v2, should any v3 servers be built in the future.

With this feature, users will achieve two steps of their RDM/RIM workflow at once: deposit publications and display linked references to them on their VIVO profile. While the feature should be designed to integrate with any SWORD repository, we will use DSpace as a proof of concept integration and basis for defining required functionality.

This draft covers:

1. **Deployment context** — VIVO/Vitro Web application ARchive (WAR) layout and where HTTP requests land
2. **Existing components** to extend or reuse (Vitro webpage editing, Create-and-Link servlet pattern, Site Admin registration, permissions, file upload limits)
3. **New components** (proposed packages, class names, responsibilities)
4. **Configuration and persistence** — admin UI, endpoint records, credentials, protocol version selection
5. **Reference UI touchpoints** — Wilma theme profile page and admin screens only (custom themes out of scope for this deliverable)
6. **SWORD packaging for PDF and METS ZIP uploads** — concrete v2 behavior, including the hard-coded VIVO → Dublin Core mapping for PDF deposits
7. **High-level request/data flows** with pseudocode sketches
8. **SWORD v3 forward compatibility** — adapter hook only; no v3 client in this draft
9. **Open decisions**

Out of scope for this draft (defer to a lower-level design pass or client feedback):

1. Authoring the METS package itself — the spec accepts a METS-compliant ZIP as an upload format but does not define how the METS document is produced (see the DSpace METS SIP Profile for the reference profile) or other pre-processing steps to transform data to IR packaging specs
2. Admin-configurable or per-site VIVO → IR field mapping UI (date conversion, multi-value flattening, subfield promotion, etc.) — v0.1 ships a **hard-coded minimal** VIVO → Dublin Core subset for PDF deposits only; METS covers advanced/custom packaging
3. Full line-by-line BIBO ↔ IR metadata field mapping for every deployment ontology profile
4. IR-side ingest workflow configuration (embargo, review queues, collection policies)
5. Retrospective migration of existing IR items into VIVO
6. Automated extraction of metadata from uploaded files (PDF text/OCR parsing or reading the METS document); metadata for PDF deposits is read from the **existing VIVO publication**, not from the file
7. SWORD v3 implementation (no production v3 servers identified yet)
8. Changes to non-reference themes (`nemo`, `tenderfoot`, etc.)

9. Retrieving URL and metadata from a SWORD repository then updating VIVO profile without depositing
10. Using SWORD to deposit an SAF to DSpace
11. Validating that the selected content license matches allowed licenses for each individual IR

This spec describes integration with **any SWORD-compliant IR** via standard protocol endpoints. It does not name or assume a particular repository product unless required by the protocol itself.

Background

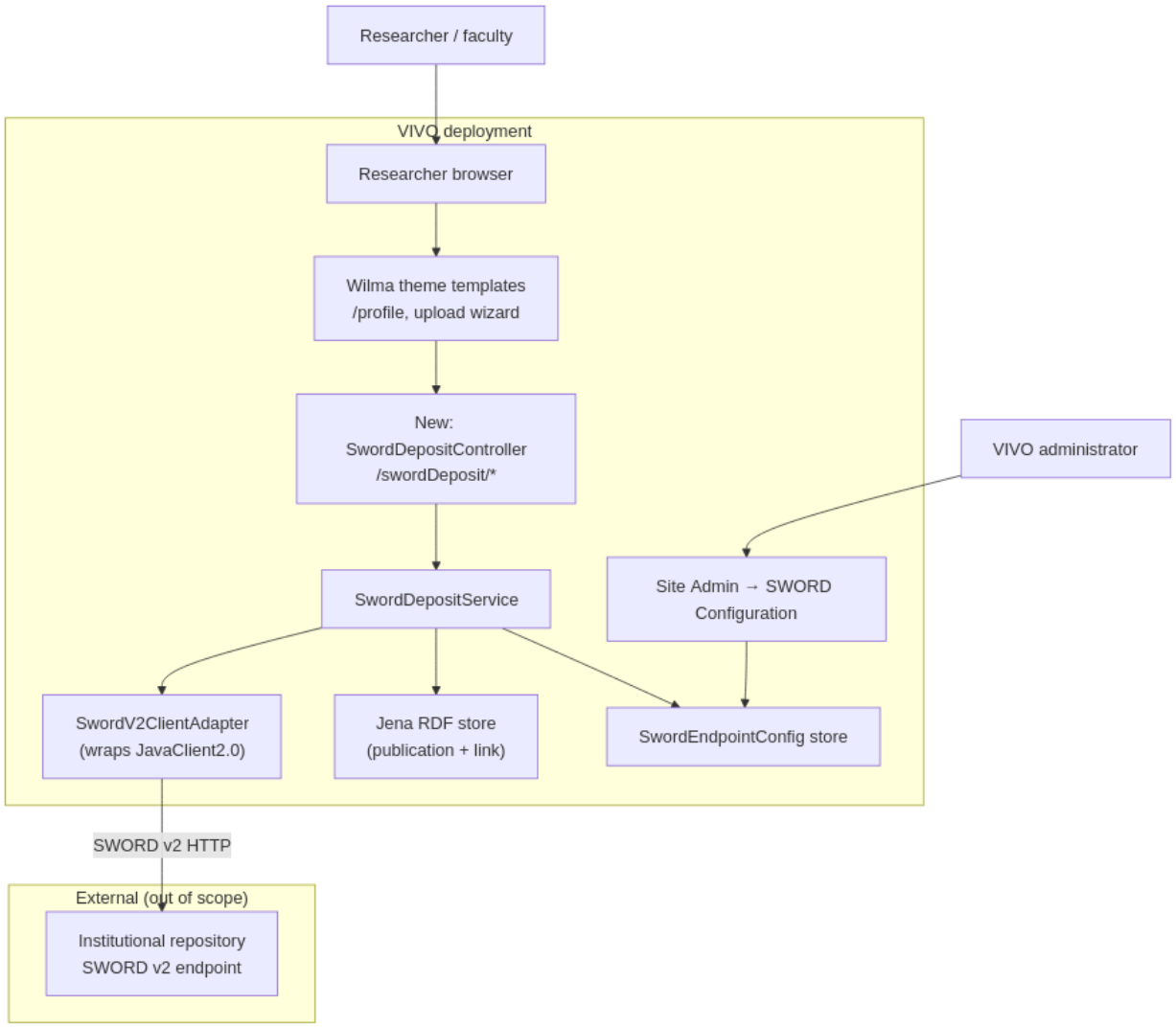
VIVO is an open-source semantic web platform for researcher profiling. Researchers and their support staff use VIVO to record and publish information about publications, grants, and professional activities. Because different research activities are represented in different repositories, a primary goal of the VIVO software is to be interoperable.

Many institutions also operate an institutional repository (IR) for long-term open-access deposit of research outputs.

Currently, researchers must deposit content in the IR separately from describing it in VIVO, then manually add the IR link to their VIVO profile. This creates duplicated effort and inconsistency. It is difficult to incentivize scholars to maintain their data in both places. If scholars could deposit their publication to their IR at the same time as linking the publication to their VIVO profile, it would be easier to ensure this data makes it to VIVO. VIVO has metadata retrieval functionality, but no external deposit integrations have been identified to date.

SWORD (Simple Web-service Offering Repository Deposit) is a standard protocol supported by several open repositories. The SWORD protocol provides a programmatic path for VIVO to initiate deposits on a researcher's behalf to a SWORD-compliant repository, such as DSpace, ArXiv, or Alma.

Actors and Roles



Actor	Role
VIVO Administrator	Registers one or more IR SWORD endpoints, credentials, default collection, protocol version (v2), and master enable flag via Site Admin.
Researcher / Faculty (VIVO User)	On an existing publication page they can edit (self-editing), clicks **Add SWORD Deposit**, chooses a configured SWORD endpoint by **Endpoint Name**, chooses a **collection** for that endpoint, selects a single PDF or METS-compliant ZIP, and submits.

Actor	Role
SWORD-enabled Repository Manager	Configures SWORD permissions and collections on the IR (outside VIVO).
Profile visitor (end user)	Follows the publication's **web link** on the public profile to the IR item (no deposit UI).
SWORD deposit module (system agent)	Validates upload; for PDF deposits, maps publication fields to Dublin Core; calls SWORD client; adds the resulting Website to the publication; logs outcome.

System Overview

Recommendation: Implement as a **VIVO `api` module feature**, use **JavaClient2.0** for SWORD v2, trigger the deposit from an existing **publication individual page**, reuse Vitro's **webpage-editing machinery** (the same AddEditWebpageFormGenerator / faux-property webpageInfoContext used to add a "Website" by hand) to write the resulting link back to the publication, and register admin screens through `BaseSiteAdminController.registerSiteConfigData`. Limit **reference UI** changes to the **Wilma** theme templates under `webapp/themes/wilma/`.

Typical URL Shapes

Assume host `https://vivo.example.edu` and profile `https://vivo.example.edu/display/n8250`:

Surface	Example URL	Notes
Individual profile (public)	`/display/{localName}`	Existing Vitro routing; deposit starts here on a **publication** page
Add Website (manual precedent)	`/editRequestDispatch?subjectUri={pubUri}&predicateUri=obo:ARG_2000028&editForm=…AddEditWebpageFormGenerator&fauxContextUri=…webpageInfoContext&rangeUri=vcard:URL`	Built-in Vitro form for adding a webpage; VIVO replicates its result programmatically (full URIs in Integration Architecture)

Surface	Example URL	Notes
Proposed deposit entry	`/swordDeposit/upload?subjectUri={publicationUri}`	GET → deposit dialog/panel (endpoint collection file); POST → deposit add Website
Proposed admin	`/admin/sword`	CRUD for endpoint configuration
Site Admin hub	`/siteAdmin`	Existing; new link under Site Configuration

VIVO / Vitro repositories touched

Repository	Role in SWORD feature	Expected change level
vivo-project/VIVO `api`	**Primary** — deposit servlet, services, SWORD adapter, webpage RDF writes	**Major** — new `sword` package
vivo-project/VIVO `webapp`	File-upload dialog template; **Wilma** publication-page button	**Moderate**
vivo-project/VIVO `home`	Default `runtime.properties` keys; optional first-time RDF for permissions	**Minor**
vivo-project/Vitro `api`	Site Admin registration hook; optional new `SimplePermission`	**Minor**
vivo-project/Vitro `webapp`	Shared `siteAdmin-siteConfiguration.ftl` only if link is registered from Vitro side	**Optional minor**
swordapp/JavaClient2.0	SWORD v2 HTTP client library	**Dependency** (Maven)

Not required: Vitro core changes beyond permission registration unless admin UI is implemented entirely in Vitro (recommended: VIVO `api` registers its own Site Admin link at startup, same pattern as other extensions).

Integration Architecture

Existing components to reuse

VIVO has **no SWORD code today**. The feature is assembled from two existing capabilities: Vitro's built-in **"Add a Website"** editing (to link the deposit back to the publication) and the **Create-and-Link** servlet/permission pattern.

Add-a-Website (webpage) editing

Adding a "Website" to an individual is a built-in Vitro edit, reached from the individual page via `editRequestDispatch`. VIVO replicates this action programmatically after a deposit. The relevant URIs (captured from a live VIVO instance, adding a webpage to a publication) are:

Parameter	Value
`predicateUri`	`http://purl.obolibrary.org/obo/ARG_2000028` (individual → webpage/contact node)
`fauxContextUri`	`http://vitro.mannlib.cornell.edu/ns/vitro/siteConfig/webpageInfoContext`
`domainUri`	`http://purl.obolibrary.org/obo/IAO_0000030` (information content entity)
`rangeUri`	`http://www.w3.org/2006/vcard/ns#URL` (vcard:URL)
`editForm`	`edu.cornell.mannlib.vitro.webapp.edit.n3editing.configuration.generators.AddEditWebpageFormGenerator`

The form fields are **URL Type**, **URL**, and **Webpage Name** (see Post-deposit linking for how the deposit result maps onto them). Reuse this N3-editing machinery rather than hand-writing RDF where practical; the exact mechanism and triple shape need a VIVO SME (Gap G-15).

Create-and-Link

Component	Location	Reuse for SWORD deposit
`CreateAndLinkResourceController`	`VIVO/api/.../CreateAndLinkResourceController.java`	**Template** for a self-editing–gated servlet that acts on an individual and writes RDF
Profile/individual UI claim buttons	`webapp/themes/wilma/templates/`	Precedent for adding a self-editing control to an individual page
`createAndLink.providers`	`runtime.properties`	Precedent for feature toggles; add `swordDeposit.enabled`

Create-and-Link already requires `SimplePermission.EDIT_OWN_ACCOUNT` and adds external URLs via `vcard:hasURL`. SWORD deposit reuses this: the **IR item URI** from the deposit receipt becomes a **Website (vcard webpage)** on the existing publication - no new `bibo:` publication is created.

Site Admin and permissions

Component	Location	Reuse for SWORD deposit
`BaseSiteAdminController.registerSiteConfigData`	`Vitro/api/.../BaseSiteAdminController.java`	Register **`manageSwordEndpoints`** → `/admin/sword` under Site Configuration
`SimplePermission`	`Vitro/api/.../SimplePermission.java`	Add **`ManageSwordEndpoints`** (admin)
`simple_permissions_admin.n3`	`Vitro/home/.../accessControl/firsttime/`	Grant new permission to Admin role set
`siteAdmin-siteConfiguration.ftl`	`Vitro/webapp/.../siteAdmin-siteConfiguration.ftl`	Renders registered config links automatically

Configuration and file handling

Component	Location	Reuse for SWORD deposit
`ConfigurationProperties` / `runtime.properties`	`Vitro/home/.../example.runtime.properties`	Global limits: `fileUpload.maxFileSize`, `fileUpload.allowedMIMETypes` (already documents `application/pdf`)
Multipart request handling	`EditConfigurationVTwo`, Vitro filters	Existing Commons FileUpload pipeline for form posts
`RDFService` / `ChangeSet`	Vitro API	Atomic write of new publication triples

Theming

Component	Location	Reuse for SWORD deposit
`ThemeInfoSetup`	`Vitro/api/.../ThemeInfoSetup.java`	Themes live under `/themes/{name}/`; Site Information selects active theme
Wilma individual page template	`VIVO/webapp/themes/wilma/templates/` (the template rendering a **publication** individual page; exact file per Gap G-15)	**Only theme modified** in v0.1

Proposed new components

New Java package: `org.vivoweb.webapp.sword`

Mirror the Create-and-Link layout under `VIVO/api/src/main/java/org/vivoweb/webapp/`:

```
None
VIVO/api/src/main/java/org/vivoweb/webapp/sword/
  controller/
    SwordDepositController.java          # @WebServlet /swordDeposit/*
    SwordAdminController.java            # @WebServlet /admin/sword/*
  service/
    SwordDepositService.java             # orchestration: deposit → add Website to
publication
    SwordEndpointConfigService.java      # load/save/test endpoint configs
    PublicationWebpageWriter.java        # add a vcard Website (webpage) to the
existing publication
    PublicationDcMapper.java             # hard-coded VIVO publication → Dublin
Core for PDF deposits
    SwordDepositAuditLog.java            # timestamp, user, collection, IR URI
  upload/
    UploadPackage.java                   # PDF or METS ZIP wrapper (stream,
filename, md5)
    UploadPackageParser.java             # validate MIME, classify PDF vs ZIP (no
content parsing)
  sword/
    SwordProtocolAdapter.java            # interface: service doc, deposit, status
    SwordV2ClientAdapter.java            # wraps org.swordapp.client.SWORDClient
    SwordV3ClientAdapter.java            # stub / NotImplemented in v0.1
    SwordAdapterFactory.java             # select by configured version (+ future
auto-detect)
    SwordPackageBuilder.java             # build Deposit; PDF → file + EntryPart
DC from publication, ZIP → setPackaging(METS)
    SwordDepositResult.java              # IR item URI, status IRI, in-progress
flag
  model/
    SwordEndpointConfig.java             # endpoint DTO
    SwordEndpointConfigStore.java        # persistence interface
    FileSwordEndpointConfigStore.java    # JSON in VIVO home (v0.1 default)
  setup/
    SwordSiteAdminSetup.java             # ServletContextListener →
registerSiteConfigData
    SwordPermissionSetup.java            # register SimplePermission + n3 snippet
```

Class responsibilities (sketch)

`SwordDepositController` — HTTP entry for researchers:

```java
@WebServlet(name = "SwordDeposit", urlPatterns = {"/swordDeposit/*"})
public class SwordDepositController extends FreemarkerHttpServlet {
    // Gated by edit access to the publication (self-editing), same policy as
other edits.
    public static final AuthorizationRequest REQUIRED_ACTIONS =
        SimplePermission.EDIT_OWN_ACCOUNT.ACTION;

    // GET  /swordDeposit/upload?subjectUri={publicationUri}
    //      → deposit dialog/panel: endpoint select
(configService.getEnabledEndpoints(...))
    //        + collection select (from that endpoint's Service Document;
pre-select
    //          defaultCollectionHref when set) + single file input (PDF or ZIP) +
OK
    // POST /swordDeposit/deposit
    //      → parse multipart (endpointId + collectionHref + file), classify PDF
vs ZIP,
    //        run SwordDepositService.deposit(...), then add Website using the
    //        selected endpoint's websiteLinkText
}
```

`SwordV2ClientAdapter` — thin wrapper over JavaClient2.0:

```java
public class SwordV2ClientAdapter implements SwordProtocolAdapter {
    private final SWORDClient client;

    public ServiceDocument fetchServiceDocument(SwordEndpointConfig cfg) {
        AuthCredentials auth = new AuthCredentials(cfg.getUsername(),
cfg.getPassword());
        return client.getServiceDocument(cfg.getServiceDocumentUrl(), auth);
    }

    public SwordDepositResult deposit(SWORDCollection collection, Deposit deposit,
                                      SwordEndpointConfig cfg) {
        DepositReceipt receipt = client.deposit(collection, deposit,
authFor(cfg));
        return SwordDepositResult.fromReceipt(receipt);
    }
}
```

`SwordDepositService` — core orchestration:

```java
public SwordDepositResult deposit(VitroRequest vreq, UploadPackage pkg,
                                  String endpointId, String collectionHref,
                                  String publicationUri) {
    UploadPackage parsed = uploadPackageParser.parse(pkg);          // validate
MIME, classify PDF vs ZIP

    // Researcher picked an enabled endpoint + collection in the deposit form.
    SwordEndpointConfig endpoint = configService.getEnabledEndpoint(endpointId);
    SWORDCollection collection = adapter.resolveCollection(endpoint,
collectionHref);

    // Fork on upload type: PDF → file + DC from the publication; ZIP → METS
package (pass-through).
    Deposit swordDeposit = packageBuilder.build(parsed, publicationUri, endpoint);

    SwordDepositResult result = adapter.deposit(collection, swordDeposit,
endpoint);

    // Link the deposit back to the existing publication as a Website (vcard
webpage).
    // Webpage Name comes from the selected endpoint's Website Link Text.
    if (result.hasItemUri()) {   // success, or pending with an item URI already
assigned
        webpageWriter.addWebsite(publicationUri, result.getItemUri(),
                                 endpoint.getWebsiteLinkText());
    }
    auditLog.record(vreq, endpoint, collectionHref, result);
    parsed.dispose();  // delete temp files
    return result;
}
```

`SwordSiteAdminSetup` — register admin nav (runs at startup):

```java
BaseSiteAdminController.registerSiteConfigData(
    "manageSwordEndpoints",
    "/admin/sword",
    null,
    SimplePermission.MANAGE_SWORD_ENDPOINTS.ACTION);
```

Files to modify (existing)

File	Change
`VIVO/api/pom.xml`	Add dependency on `org.swordapp:sword-client` (JavaClient2.0 coordinates TBD — may require vendoring or JitPack)
`VIVO/webapp/themes/wilma/templates/` (publication individual page; exact file per Gap G-15)	Add **Add SWORD Deposit** control in the **Websites** section (below Websites, above the metadata tabs) when `swordDeposit.enabled=true`
`VIVO/home/.../example.runtime.properties`	Document new keys (see Configuration)
`Vitro/.../SimplePermission.java`	Add `MANAGE_SWORD_ENDPOINTS`
`Vitro/.../simple_permissions_admin.n3`	Include new permission for Admin role

Configuration model

Two tiers align with the behavior scenarios and VIVO conventions:

Tier 1 — System administrator (`runtime.properties`)

Global feature flags and limits. Not editable in the Site Admin UI.

Key	Example	Purpose
`swordDeposit.enabled`	`true`	Master switch; hides UI and rejects `/swordDeposit/*` when false
`swordDeposit.configStore`	`file`	`file` (default) or future `rdf`
`swordDeposit.configPath`	`{vivoHome}/config/sword-endpoints.json`	Endpoint registry file location

Key	Example	Purpose
`swordDeposit.maxUploadBytes`	`52428800`	Override; default falls back to `fileUpload.maxFileSize`
`swordDeposit.allowedMimeTypes`	`application/pdf,application/zip`	Upload validation
`swordDeposit.defaultProtocolVersion`	`v2`	Used when admin leaves version as `auto`
`swordDeposit.tempDir`	`{java.io.tmpdir}/vivo-sword`	Short-lived upload staging

Tier 2 — VIVO administrator (Site Admin UI)

Stored in **`sword-endpoints.json`** as an **array of endpoint records**. Admins may configure one or more SWORD endpoints at **Site Admin → Site Configuration → SWORD Configuration** (`/admin/sword`).

Each `SwordEndpointConfig` record must make at least these fields configurable in the admin UI (other fields below remain supported):

Admin label	Field	Type	Required	Description
Endpoint Name	`displayName`	string	Yes	Label shown in the researcher's deposit form endpoint `<select>`
Endpoint Service Document URL	`serviceDocumentUrl`	URL	Yes	SWORD v2 Service Document URL
Username	`username`	string	Cond.	Basic auth user
Password	`password`	secret	Cond.	Encrypted at rest (see D-02)

Website Link Text	`websiteLinkText`	string	Yes	Webpage Name written on the publication after a successful deposit to this endpoint (e.g. `Institutional Repository`, `SWORD Deposit`)

Additional per-endpoint fields:

Field	Type	Required	Description
`id`	UUID	Yes	Stable key (value of the deposit form `<select>` option)
`enabled`	boolean	Yes	Per-endpoint enable; only enabled endpoints appear in `configService.getEnabledEndpoints(...)`
`protocolVersion`	enum	Yes	`v2` (default), `v3` (disabled until implemented), `auto` (future: probe endpoint)
`authType`	enum	Yes	`basic` (v0.1 only)
`defaultCollectionHref`	URL	No	Collection IRI/href from the Service Document; pre-selected in the researcher's deposit-form collection `<select>` when set (researcher may still choose another collection for that endpoint)
`defaultLicenseUri`	URI	No	Passed in Atom metadata if IR requires (out of scope: validating that the license is accepted in the IR)
`slugPrefix`	string	No	Optional SWORD Slug prefix
`inProgressByDefault`	boolean	No	SWORD `In-Progress` header default

Save / test actions:

- **Test connection** — GET Service Document, populate collection dropdown, surface HTTP/auth errors to admin (implements BS01 validation steps).
- **Version auto-detect (future)** — `auto` attempts v2 Service Document first; if v3 signposting is detected, set adapter to v3 when implemented.

Researcher permissions

Permission	Who	Purpose
`EditOwnAccount`	Researcher (existing)	Gate the **Add SWORD Deposit** control - same as Create-and-Link
Self-editing policy	Researcher (existing)	User may only deposit for / add a Website to publications they are allowed to edit
`ManageSwordEndpoints`	VIVO admin (new)	Configure endpoints

No new researcher role; reuse **`EditOwnAccount`** plus existing self-editing configuration. Add **`ManageSwordEndpoints`** for administrators only.

Reference UI touchpoints (Wilma theme)

Location	Change
Publication individual page (Wilma template; exact file per Gap G-15)	When signed in and able to edit the publication, show an **Add SWORD Deposit** button in the **Websites** section (below Websites, above the metadata tabs). Opens the deposit dialog/panel (GET `/swordDeposit/upload?subjectUri={pubUri}`).
`swordDepositUpload.ftl` (new)	Deposit **dialog or panel** (not an immediate OS file picker). Form fields: (1) **endpoint** `<select>` listing enabled endpoints by **Endpoint Name** (`displayName`), from `configService.getEnabledEndpoints(...)`; (2) **collection** `<select>` listing collections from the selected endpoint's Service Document (same source as admin **Test connection**), pre-selecting `defaultCollectionHref` when set (or the sole collection when only one exists); (3) single-select **file** input (`accept=".pdf,.zip"`) with short help for PDF vs METS ZIP; (4) **OK** button that POSTs to `/swordDeposit/deposit` with `endpointId` `collectionHref` file. Changing the endpoint refreshes the collection list. If only one endpoint is enabled, pre-select it.
	Single-select file input (`accept=".pdf,.zip"`), short help text describing the two accepted formats: a PDF (VIVO publication fields sent as Dublin Core with the deposit) or a METS-compliant ZIP (metadata inside the package). Optional IR endpoint/collection picker only when more than one is enabled; submit.
`swordDepositResult.ftl` (new)	Success → the new **Website** on the publication (label selected endpoint's **Website Link Text**) link to the IR item; failure → actionable error (Error workflow is gap G-06)

Location	Change
Publication individual page (Wilma template; exact file per Gap G-15)	When signed in and able to edit the publication, show an **Add SWORD Deposit** button in the **Websites** section (below Websites, above the metadata tabs). Opens the deposit dialog/panel (GET `/swordDeposit/upload?subjectUri={pubUri}`).
`swordDepositUpload.ftl` (new)	Deposit **dialog or panel** (not an immediate OS file picker). Form fields: (1) **endpoint** `<select>` listing enabled endpoints by **Endpoint Name** (`displayName`), from `configService.getEnabledEndpoints(...)`; (2) **collection** `<select>` listing collections from the selected endpoint's Service Document (same source as admin **Test connection**), pre-selecting `defaultCollectionHref` when set (or the sole collection when only one exists); (3) single-select **file** input `(accept=".pdf,.zip"`) with short help for PDF vs METS ZIP; (4) **OK** button that POSTs to `/swordDeposit/deposit` with `endpointId` `collectionHref` file. Changing the endpoint refreshes the collection list. If only one endpoint is enabled, pre-select it.
`admin/swordEndpointList.ftl` (new)	List/add/edit/delete **multiple** endpoint records
`admin/swordEndpointEdit.ftl` (new)	Form for Tier 2 fields (including Endpoint Name, Service Document URL, Username, Password, Website Link Text) + **Test connection**

Shared templates under `webapp/templates/freemarker/` may be included from Wilma if other themes later opt in; v0.1 only ships Wilma overrides.

i18n: Add keys to `themes/wilma/i18n/` (and optionally `webapp/i18n/`) following existing VIVO property file conventions.

Uploads, metadata, and packaging

VIVO accepts two upload formats and does not assume a vendor-specific archive layout. Producing a METS package is out of scope; VIVO only accepts it as an upload format. Metadata for PDF deposits is read from the **existing VIVO publication**; metadata for METS ZIP deposits stays inside the package.

Accepted uploads

Input	Validation	Deposited to IR as
`application/pdf`	Single PDF within size limit	File \ Atom `EntryPart` with Dublin Core from the publication (hard-coded VIVO → DC mapping); no **packaging**
`application/zip`	Valid ZIP within size limit; treated as a **METS package** and passed through unmodified (VIVO does not parse, validate, or repackage its contents)	Package deposit with `Packaging` set to the METS SIP profile URI (see **SWORD v2 package construction** below). VIVO does **not** inject publication fields into the ZIP.

Rejected: disallowed MIME types; uploads exceeding size limits.

VIVO→ Dublin Core mapping (PDF deposits only)

For PDF deposits, VIVO reads a **small hard-coded set** of properties from the selected publication individual and attaches them to the SWORD deposit as Dublin Core terms via JavaClient2.0 `EntryPart.addDublinCore(...)`. This mapping is **not** exposed in the Site Admin UI in this revision (date conversion, multi-value flattening, subfield promotion, and per-site overrides are left to implementation / later work). Sites that need packaging or metadata beyond this subset should use a **METS-compliant ZIP**.

Initial mapping (direction: **from VIVO to DC for deposit**):

VIVO / BIBO source	Dublin Core (deposit)	Notes

`bibo:title` (fallback `rdfs:label`)	`dc:title`	Primary sample field; see package-construction sketch below
`bibo:authorList` → `foaf:Person` / `vivo:Authorship` nodes	`dc:creator`	**Implementation note:** flatten person labels to one or more `dc:creator` values when present; exact traversal left to implementers
`bibo:date` (Vitro date precision)	`dc:date`	**Implementation note:** convert VIVO date precision to a DC-appropriate string
`bibo:doi`	`dc:identifier`	Prefer a DOI URI or bare DOI string as the IR expects
`bibo:abstract`	`dc:description`	
`rdf:type` (`bibo:Article`, `bibo:Book`, …)	`dc:type`	Map to a simple type label when useful

Implementation notes (not in the sample code):

- Omit a DC element when the VIVO source value is absent rather than sending an empty element.
- Multi-valued VIVO fields (e.g. authors) may yield repeated `addDublinCore` calls; implementation should flatten fields as appropriate.

A fuller IR-specific mapping remains Gap **G-01**; this version implements this **minimal DC subset** for PDF deposits only.

METS package (ZIP uploads)

A ZIP upload is treated as a **METS-compliant Submission Information Package (SIP)** and deposited to the IR **as-is**. VIVO does not parse, validate, or repackage the archive, and this spec does not define how the METS document is authored — it only defines that a METS ZIP is an accepted upload format.

- On deposit, the SWORD client sets the `Packaging` value to the METS SIP profile URI (see **SWORD v2 package construction** below).

- Reference profile: DSpace METS SIP Profile. The feature is designed to be IR-agnostic, but DSpace is the reference/target IR in practice.
- Producing a valid METS package is the responsibility of the depositor or an upstream tool and is out of scope here.
- **Advanced / custom metadata**: use METS when the hard-coded PDF VIVO → DC mapping is insufficient; do not expect an Admin mapping UI in this revision.

Post-deposit linking (add a Website to the publication)

The publication already exists in VIVO, so **no publication metadata is entered** as part of the deposit (PDF deposits read selected fields for the IR; they do not write new publication fields). On a successful (or pending-with-URI) deposit, VIVO adds a **Website (vcard webpage)** to the existing publication using Vitro's webpage-editing machinery (see Add-a-Website (webpage) editing). The example payload maps onto the built-in webpage form fields:

Field (webpage form)	Value	vcard / VIVO target
URL Type	`URL`	webpage type on the `vcard:URL` node
URL	`{IR item URI from the deposit receipt}`	`vcard:url`
Webpage Name	`{selected endpoint's websiteLinkText}`	link label (`rdfs:label`) on the `vcard:URL` node

The **Webpage Name** is taken from the **Website Link Text** of the SWORD endpoint the researcher selected in the deposit form (not a hard-coded `"SWORD Deposit"` string). That lets each configured IR present a distinct label on the publication (e.g. `Campus IR`, `Disciplinary Repository`).

The triple shape follows Vitro's standard webpage model: publication `-obo:ARG_2000028→` webpage/contact node `-vcard:hasURL→ vcard:URL`. The exact triples are written by reusing the `AddEditWebpageFormGenerator` machinery rather than hand-rolling RDF; confirming the precise mechanism and triple shape is Gap G-15.

Design rationale: why a Website (and not an identifier field)

A SWORD deposit returns a **location** (the item's landing page in the IR, e.g. a DSpace item / Handle URL), not an **identifier** in a governed scheme. VIVO models these differently, and `vcard:hasURL` (the "Website" relation) is its purpose-built home for "a URL related to this thing" — which is exactly what a deposit landing page is.

The Identity-tab identifier fields (DOI, PubMed ID, etc.) are the wrong home because:

- **Semantically false** - an IR URL is not a DOI; storing it as bibo:doi misrepresents the data and can pollute metadata exports/OAI harvesting.
- **Rendered through fixed resolvers** - those fields link out through scheme proxies (`doi.org/{value}`, PubMed, …) and the public "GET IT" button assumes a registered identifier, so a non-DOI value produces a broken link (confirmed in testing).
- **Type-dependent** - available identifier properties vary by publication class (Abstract, Academic Article, Case Study, Editorial…), which would force a fragile per-type mapping.

By contrast, a Website is semantically correct, **available uniformly on any publication type**, consistent with VIVO's existing Create-and-Link pattern (which already attaches external URLs via `vcard:hasURL`), carries a human-readable label (the endpoint's **Website Link Text**), is non-destructive (a publication may hold several — useful when multiple IR endpoints are configured), and needs **no custom ontology term or link resolver** — it renders as a plain, clickable link out of the box.

The one tradeoff is **prominence**: an identifier can surface as a bold "GET IT" full-text button, whereas a Website appears in the Websites list. If a first-class "full text available" affordance is later desired, the clean approach is a dedicated property plus theme-level rendering — treated as a possible revision-phase enhancement, not a reason to overload an identifier field. Likewise, if an IR mints a real DOI or Handle for the deposit, that genuine identifier belongs in the corresponding field; it is a different value from the landing URL, which remains a Website.

SWORD v2 package construction

The deposit package is determined by the **upload type** (not by an Admin mapping UI):

Upload	JavaClient2.0 usage
PDF	`Deposit` with the PDF `InputStream`, `mimeType=application/pdf`, optional `slug`, `md5` digest, plus an `EntryPart` populated with Dublin Core from the publication via `addDublinCore(...)`. **No packaging** URI.

Upload	JavaClient2.0 usage
METS ZIP	`Deposit` with the ZIP `InputStream`, `mimeType = application/zip`, and `setPackaging(...)` set to the METS SIP profile URI. The archive is sent unmodified (no VIVO → DC injection).

Sample fork (PDF path uses `EntryPart.addDublinCore` as in the JavaClient2.0 README; METS path mirrors the JavaClient2.0 METS deposit test):

```java
// DSpace METS SIP packaging identifier, as used by the JavaClient2.0 METS test
private static final String METS =
"http://purl.org/net/sword/package/METSDSpaceSIP";

Deposit deposit = new Deposit();
deposit.setFile(parsed.getInputStream());
deposit.setFilename(parsed.getFilename());
deposit.setMd5(parsed.getMd5());               // optional Content-MD5

if (parsed.isPdf()) {
    deposit.setMimeType("application/pdf");
    // PDF path: attach Dublin Core from the existing VIVO publication (hard-coded
mapping).
    // Sample shows title only; other fields from the VIVO → DC table follow the
same pattern.
    String title = publicationDcMapper.titleFrom(publicationUri); // bibo:title
(fallback rdfs:label)
    if (title != null && !title.isBlank()) {
        EntryPart ep = new EntryPart();
        ep.addDublinCore("title", title);
        deposit.setEntryPart(ep);
    }
} else { // METS ZIP — advanced/custom packaging; metadata already in the archive
    deposit.setMimeType("application/zip");
    deposit.setPackaging(METS);
}
```

The **`Packaging`** URI signals to the IR that the ZIP is a METS SIP. The feature targets any SWORD-compliant IR, but DSpace is the reference implementation, so the DSpace METS SIP

identifier (**`http://purl.org/net/sword/package/METSDSpaceSIP`**, per the DSpace METS SIP Profile) is used by default.

Collection **`accept`** and **`acceptPackaging`** from the Service Document drive validation before POST (JavaClient2.0 already warns on mismatch).

Storage: Streams only; if spooled to disk, use `swordDeposit.tempDir` and delete in `finally`.

SWORD v2 protocol surface (implementation mapping)

Operation	SWORD v2	Adapter method
Service Document	`GET` Service-Document IRI	`fetchServiceDocument`
List collections	Parse workspaces from Service Document	`listCollections`
Deposit	`POST` to Collection IRI	`deposit` → `DepositReceipt`
In-progress / complete	`In-Progress` header; Status-IRI in receipt	`pollStatus` (optional v0.1)
Auth	HTTP Basic	`AuthCredentials`

Headers mapped by JavaClient2.0 (parent BS02): **Authorization**, **Content-Type**, **Content-Length**, **Slug**, **In-Progress**, **Digest** (when MD5 supplied).

In-progress deposits

If the IR returns **`In-Progress: true`**:

- v0.1: Show researcher a **"deposit submitted, pending IR review"** message; still create the VIVO publication stub with Status-IRI stored as a **`vivo:DepositStatus`** annotation (new optional predicate) or admin-visible log only (see D-04).
- Later: background poller on Status-IRI until `true` → update webpage link when item URI available.

SWORD v3 forward compatibility

Concern	v0.1 approach
Adapter selection	`SwordAdapterFactory.forVersion(config.getProtocolVersion())`
v3 implementation	`SwordV3ClientAdapter` throws `UnsupportedOperationException`
Admin `auto` mode	Attempt v2 Service Document; if failure and v3 client present, try v3 (disabled until client exists)
Auth	Document OAuth bearer for v3 in config schema; unused in v0.1

Data flow: happy path

Happy-path sequence (the previous diagram has been removed because it predates this flow):

1. Signed-in user opens an existing **publication** page they can edit.
2. In the **Websites** section they click **Add SWORD Deposit**.
3. A deposit dialog/panel opens with: an endpoint `<select>` (enabled endpoints by **Endpoint Name** from `configService.getEnabledEndpoints(...)`), a collection `<select>` (from that endpoint's Service Document; pre-select `defaultCollectionHref` when set), a file input, and **OK**.
4. They choose an endpoint and collection, choose a single file — a PDF or a METS-compliant ZIP — and click **OK**.
5. VIVO deposits the file to the selected collection via SWORD v2 (PDF → file Dublin Core from the publication; ZIP → METS packaging, unmodified).
6. The IR returns a receipt/status document containing the new IR item URI.
7. VIVO adds a **Website** to the publication: `{ URL Type: URL, URL: <IR item URI>, Webpage Name: <selected endpoint's Website Link Text> }`.
8. The user sees a success (or “deposit submitted, pending IR review”) message linking to the new Website and the IR item.

Behavior Scenarios

BS01: Administrator configures a SWORD endpoint in VIVO

Step	Description
Given	The user has VIVO Administrator access.
	A target IR with SWORD (v2 or v3) enabled exists and credentials are available.
When	The administrator navigates to the SWORD configuration screen in the VIVO admin interface.
	The administrator enters the required configuration fields and saves.
Then	The VIVO SWORD client retrieves and validates the Service Document from the endpoint URL.
	Available collections in the IR are retrieved and listed for selection as the default.
	The Deposit to Repository option becomes available in the VIVO publication editing interface for users of that VIVO instance.

BS02: Researcher selects a publication to deposit

Step	Description
Given	A researcher is logged into VIVO.
	SWORD deposit is configured and enabled for the VIVO instance.
	No IR record exists yet for the publication being added.
	The IR supports SWORD v2 Server.
When	The researcher navigates to a publication entry.
And	The researcher selects the (new) button “**SWORD Deposit**”
Then	A pop up appears to upload a file.
When	The researcher selects a PDF or ZIP file from their computer and clicks upload.

BS03: Researcher deposits to a SWORD v2 server

Step	Description
Given	A researcher is logged into VIVO and has selected a valid PDF or ZIP file to deposit.

Step	Description
	SWORD deposit is configured and enabled for the VIVO instance.
	No IR record exists yet for the publication being added.
	The IR supports SWORD v2 Server.
When	The researcher clicks upload
Then	The SWORD client constructs a deposit package in the configured format.
	The package is submitted to the IR SWORD endpoint.
	The IR returns a deposit receipt / status document containing the URI of the new IR item.
	The user is notified of the deposit status (successful or in progress) in the GUI.
	VIVO adds a Website (a vcard webpage) to the publication whose URL is the IR item URI returned by the deposit.
	The deposit is logged with timestamp, depositing user, target collection, status, and IR item URI in the VIVO web server logs.

Error Scenarios

ID	Condition	User-visible behavior	Log / admin
ES01	Feature disabled or no endpoints	Hide upload button; 404 on direct URL	—
ES02	IR HTTP error / SWORD error document	Message with IR status	Full response body in audit log
ES03	Auth failure (401/403)	“Cannot authenticate to repository — contact administrator”	Never log password
ES04	MIME / packaging rejected	Explain accepted types from collection metadata	—
ES06	Publication already exists in IR	Gap G-06	Gap G-06
ES07	Deposit fails for any other reason	Gap G-06	Gap G-06

Other Requirements

Documentation

- How to configure your VIVO to enable SWORD deposits
- VIVO ontology to DSpace METS profile mapping to support users creating METS files for deposit
- Required data
- Troubleshooting
- Governance, support, and contact information
- Accessibility features and compliance
- Reference documentation, such as field definitions

The documentation may include screenshots and examples.

Security and Data Privacy

- SWORD deposit is HTTPS with authentication

Performance and Scalability

The SWORD protocol includes many helpful responses, including in-progress, to assist with performance.

Governance

The feature design should be reviewed, further edited, and if agreed upon, formally accepted by the VIVO development team.

Maintenance of this feature after the implementation phase will fall to the VIVO development team.

Implementation Gaps

#	Gap	Description	Owner
G-01	Metadata mapping: VIVO ontology → SWORD/IR	A full field mapping from VIVO's publication ontology to the target IR's required and recommended metadata fields is needed. **Current spec**: ship a **hard-coded minimal VIVO → Dublin Core** subset for **PDF deposits only** (see VIVO → Dublin Core mapping); omit Admin mapping UI; use **METS** for advanced/custom packaging.	Implementation team / Metadata specialist
G-02	Configurable metadata mappings	Configurable metadata fields or mappings should be a v2.	Implementation team or v2 team
G-06	Actionable error for deposit failure	Several errors are defined in Error Scenarios. During implementation, determine a method for capturing errors if (1) a duplicate publication exists in the IR; or (2) the deposit fails for any other reason.	Implementation team
G-15	VIVO webpage-linking specifics (SME)	Confirm with a VIVO SME: (a) the exact Wilma template that renders the **publication** individual page and where to inject the **Add SWORD Deposit** control (below Websites, above the metadata tabs), and (b) the precise `vcard:URL` triple shape and whether to add the Website by driving `AddEditWebpageFormGenerator` programmatically vs writing triples via `RDFService`. Anchors captured from a live instance: predicate `obo:ARG_2000028`, faux context `webpageInfoContext`, domain `IAO_0000030`, range `vcard:URL`.	Implementation team
G-16	Generating METS file for users	As a first iteration, the feature will deposit PDF or ZIP with METS to a SWORD-enabled endpoint. How the system could help the user generate the METS file is out of scope for v1 but could be pursued in future development.	Implementation team or v2 team

#	Gap	Description	Owner
G-17	Incorporating feature in VITRO	Team recommends that the code be merged into vitro with data mappings in VIVO. The consulting team determined that as a first draft feature to test, this may create additional dependences. Splitting the code between vitro and VIVO is a potential next step v2 activity after initial implementation.	Implementation team or v2 team
G-18	Pre-deposit error detection or validation	Determine methods for helping users enter required metadata to submit a successful deposit.	Implementation team
G-19	Deletions	Add functionality to the feature for deleting a publication from the SWORD endpoint that had previously been deposited with SWORD.	Implementation team
G-20	Workflows discussion	Several questions about the IR endpoint's workflows, validation, and acceptance of data were surfaced during spec feedback meetings. VIVO team to discuss these questions and determine if they are Implementation or v2+ concerns	VIVO team
G-21	Expand functionality to multiple endpoints	Allow users to deposit multiple files to multiple endpoints. Note: In current spec, multiple files can be deposited to a single endpoint if using Zip with METS deposit pathway	v2 team

Suggested epics

Parallelizable units of work for a single delivery:

Epic	Deliverable	Repos
0 — Spike	JavaClient2.0 dependency proof; manual Service Document + deposit against a test IR	`VIVO/api`

Epic	Deliverable	Repos
1 — Admin config	Endpoint CRUD, Test connection, Site Admin link, permission	`VIVO/api`, `Vitro/api`, `Vitro/home`
2 — Upload UI	Parser (PDF vs METS ZIP classification), Wilma publication-page **Add SWORD Deposit** button deposit dialog (endpoint + collection + file + OK)	`VIVO/api`, `VIVO/webapp`
3 — Deposit + RDF	`SwordDepositService`, PDF VIVO → DC mapping (`PublicationDcMapper`), receipt handling, add **Website** to the publication (`PublicationWebpageWriter`)	`VIVO/api`
4 — Hardening	Error scenarios, duplicate detection, in-progress handling, audit log	`VIVO/api`
5 — SWORD v3	`SwordV3ClientAdapter`, OAuth, admin auto-detect	`VIVO/api` (future)